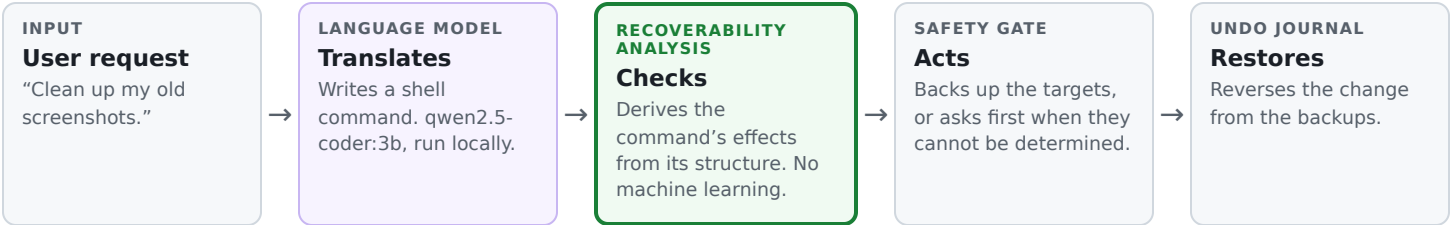


How the Recoverability Analysis Works

SynapseOS has two separate components. A local language model translates a request into a shell command; the recoverability analysis then decides, before that command runs, what it will change and whether it can be undone. **The analysis does not use the language model:** it is deterministic program analysis, specified in Chapter 3, §2.1(c) (p. 60).

Where the analysis sits



Probabilistic: the same request may yield different commands. **Deterministic:** the same command always yields the same decision.

Inside the analysis, in five steps

Running example: `find . -name "Screenshot*.png" -delete`

1

Parse
Reads the command's structure with a shell-language parser.

Program `find`; search path `.`; filter `-name "Screenshot*.png"`; action `-delete`.

2

Look up
A rule table records what each program does to files.

`find ... -delete` removes every file that matches its filter; which files those are is not named in the command.

3

Compose
Sees through searches, pipes, loops and wrappers to the program that acts.

The deletion is carried out by `find` itself on files it discovers while running. The same reasoning applies to `find -exec rm`, `| xargs rm`, `for` loops and `sh -c`.

4

Resolve
Runs only the proven read-only part, in a sandbox where nothing can be written.

ILLUSTRATION (SIMPLIFIED) OF AN INTERNAL STEP; USERS NEVER SEE IT:

```
$ find . -name "Screenshot*.png" # -delete replaced by a read-only print
./Screenshot 2026-09-20 184502.png
./Screenshot 2026-09-14 101211.png
```

5

Decide
Classifies the effects and plans what to back up.

ACTUAL ANALYSIS OUTPUT, UNEDITED:

```
command: find . -name "Screenshot*.png" -delete
composition: enabled (full algorithm)
class: recoverable-with-capture
remove ./Screenshot 2026-09-20 184502.png
remove ./Screenshot 2026-09-14 101211.png
```

Both files are backed up before the command runs. Had their effects been undeterminable (an unknown program, inline Python, `shred`), the command would be classified *unrecoverable* and confirmation required.

Why the separation matters

It cannot be talked into anything
The safety decision is made by code, not a model. It judges the command as actually written, so a flawed or unexpected command from the model is judged on its real effects.

It is independent of the command's source
A command from the model, a person, or a script is treated identically. This is why the 777-command evaluation (§3.1, p. 72) required no model at all.

It is the computer-science contribution
The analysis is static program analysis in the tradition of abstract interpretation (p. 23), not machine learning. The model is the interface; the analysis is the research.